

Библиотека Matplotlib.

Matplotlib е open-source библиотека за създаване на графики в Python. За да инсталираме библиотеката изпълняваме стандартната `pip` команда в терминал/command prompt/powershell. `pip install matplotlib` След това включването на модула в на нашето приложение може да стане с `import matplotlib` Версията на matplotlib можем да видим като принтираме атрибута **version**:

```
In [1]: import matplotlib

print(matplotlib.__version__)

3.7.1
```

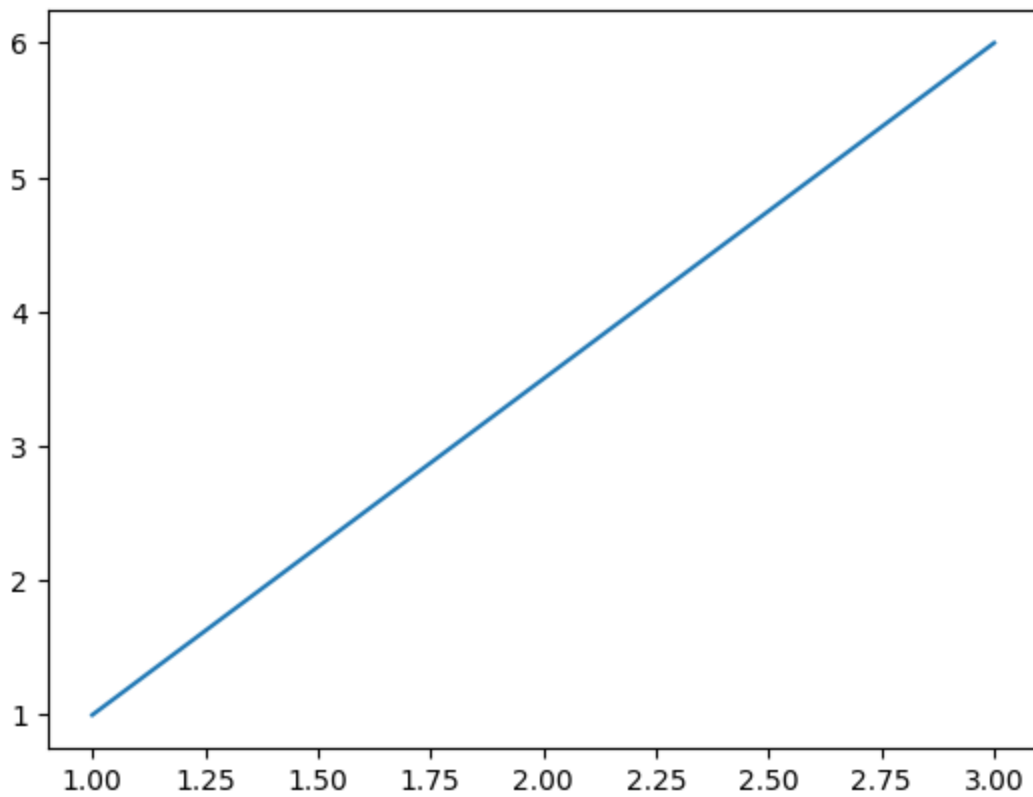
Основната функционалност на matplotlib се крие в pyplot подмодула. Както вече видяхме при други библиотеки, съществува стандарт за псевдоним на pyplot подмодула - `plt`. В долния пример използваме numpy arrays за да съхраняваме x и y координати в два отделни масива. Съответно в примера имаме две точки с координати (1, 1) и (3, 6). По подразбиране рисуваме линия между тях. Съществуват и много други възможности за промяна на визията на графиката, част от които ще разгледаме в това упражнение.

```
In [2]: import matplotlib.pyplot as plt
import numpy as np

x_coordinates = np.array((1, 3))
y_coordinates = np.array((1, 6))

plt.plot(x_coordinates, y_coordinates)

plt.show()
```



Чертане на точки

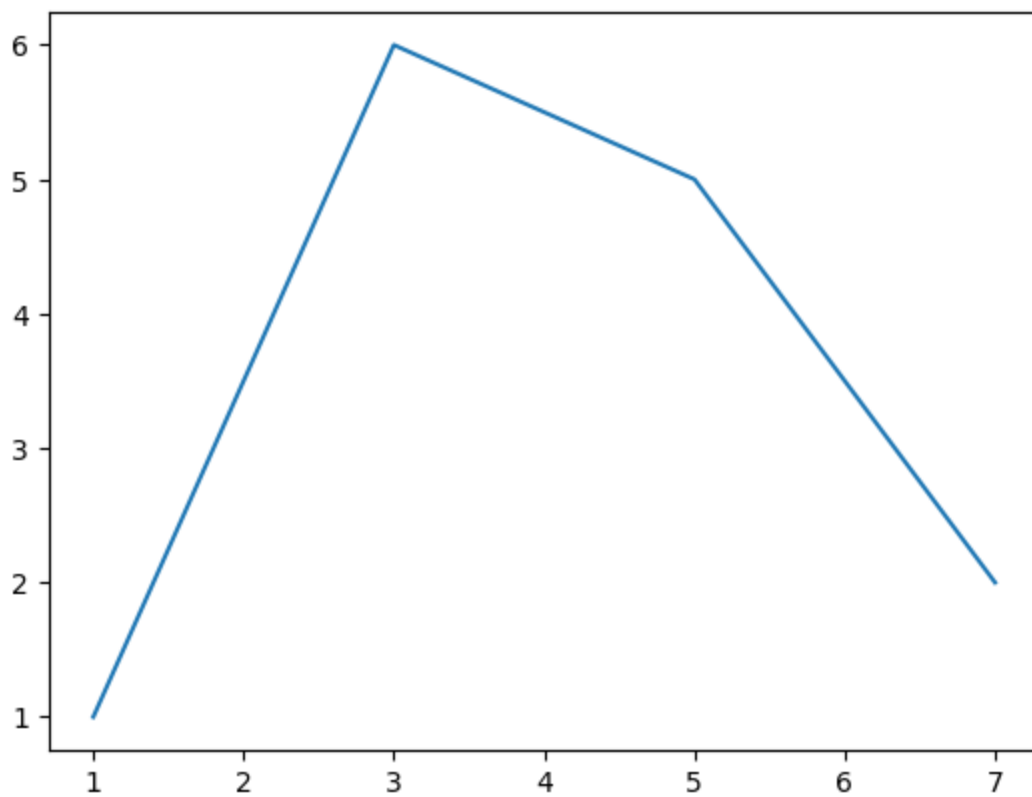
Както вече видяхме, `plot()` функцията чертае точки в диаграма. На горната графика видяхме, че по подразбиране функцията чертае линия между описаните точки. При няколко точки това ще изглежда така:

```
In [3]: import matplotlib.pyplot as plt
import numpy as np

x_coordinates = np.array((1, 3, 5, 7))
y_coordinates = np.array((1, 6, 5, 2))

plt.plot(x_coordinates, y_coordinates)

plt.show()
```



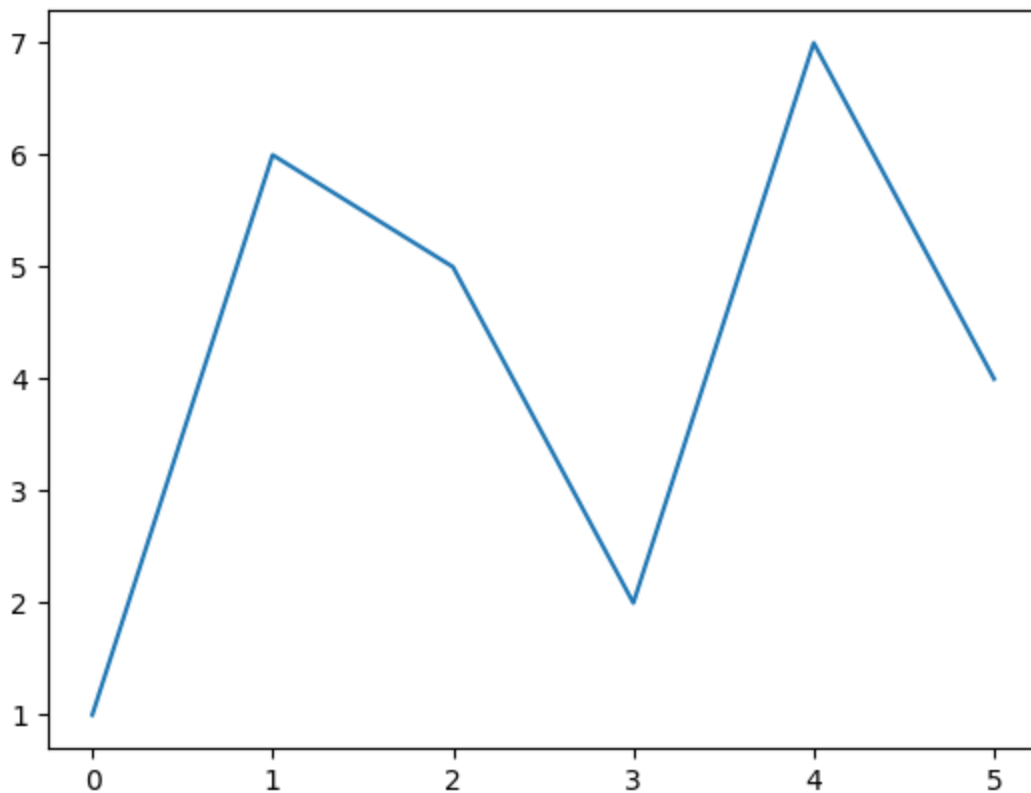
Ако не посочим стойности по абсцисата (x_coords), те придобиват стойност по подразбиране (0, 1, 2, 3...) в зависимост от броя зададени стойности по ординатата.

```
In [4]: import matplotlib.pyplot as plt
import numpy as np

y_coordinates = np.array((1, 6, 5, 2, 7, 4))

plt.plot(y_coordinates)

plt.show()
```



Промяна визията на Matplotlib графики.

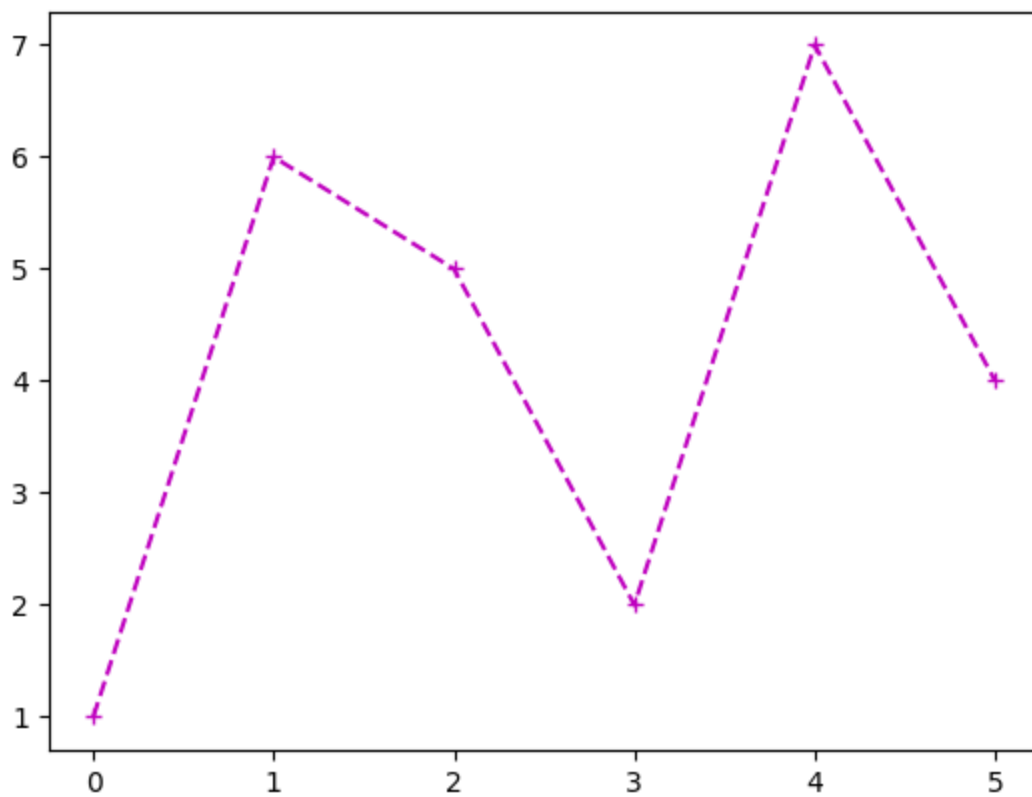
Един от най-лесните начини за промяна на визията на графики в Matplotlib е чрез формат стрингове (format strings - fmt). Те следват конвенцията 'marker line color', където marker е стилът на "маркерът"/точката, line - стилът на линията, която свързва точките и color - цвят на маркера и линията.

```
In [5]: import matplotlib.pyplot as plt
import numpy as np

y_coordinates = np.array((1, 6, 5, 2, 7, 4))

plt.plot(y_coordinates, '+--m')

plt.show()
```



Някои от най-използваните маркери са:

- 'o' - кръг
- '*' - звезда
- '.' - точка
- ',' - пиксел
- 'x' - x
- '+' - плюс
- 's' - square - квадрат
- 'D' - diamond - диамант
- 'd' - thin diamond
- 'p' - pentagon - петоъгълник
- 'h' / 'H' - hexagon - шестоъгълник

Линиите могат да бъдат:

- '-' - Плътна линия
- ':' - Dotted line
- '--' - Dashed line
- '-.' - Dashed/dotted line

Цветовете могат да бъдат:

- 'r' - червен
- 'g' - зелен

'b' - син
'c' - циан
'm' - магента
'y' - жълт
'k' - черен
'w' - бял

Освен с format string тези свойства, както и някои други могат да бъдат задавани чрез аргументи:

markersize/ms - размер на маркера
markeredgecolor/mec - цвят на ръба на маркера
markerfacecolor/mfc - цвят на лицето на маркера

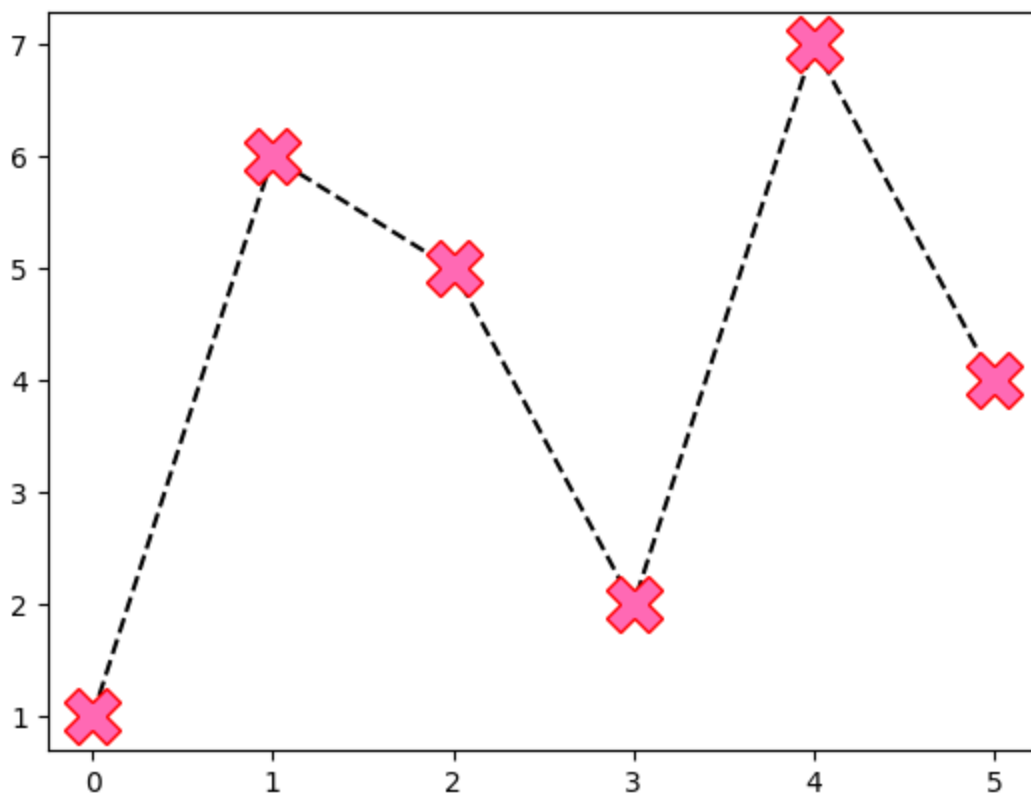
Тези аргументи поддържат и много повече цветове. Могат да се задават шестнадесетични цветови кодове или едно от 140-те поддържани имена на цветове.

```
In [6]: import matplotlib.pyplot as plt
import numpy as np

y_coordinates = np.array((1, 6, 5, 2, 7, 4))

plt.plot(y_coordinates,
         marker='X',
         color = 'black',
         linestyle='--',
         markersize=20,
         markeredgecolor='red',
         markerfacecolor='hotpink')

plt.show()
```



Именуване на графики и оси.

Именуване на оси може да направим посредством `xlabel()` и `ylabel()` функциите на `pyplot`. Именуване на графиката извършваме с `title()` функцията. Също така може да използваме речник, който да ни посочи спецификации за фонта на съответните текстове. Източник на примера:

https://www.w3schools.com/python/matplotlib_labels.asp

```
In [7]: import numpy as np
import matplotlib.pyplot as plt

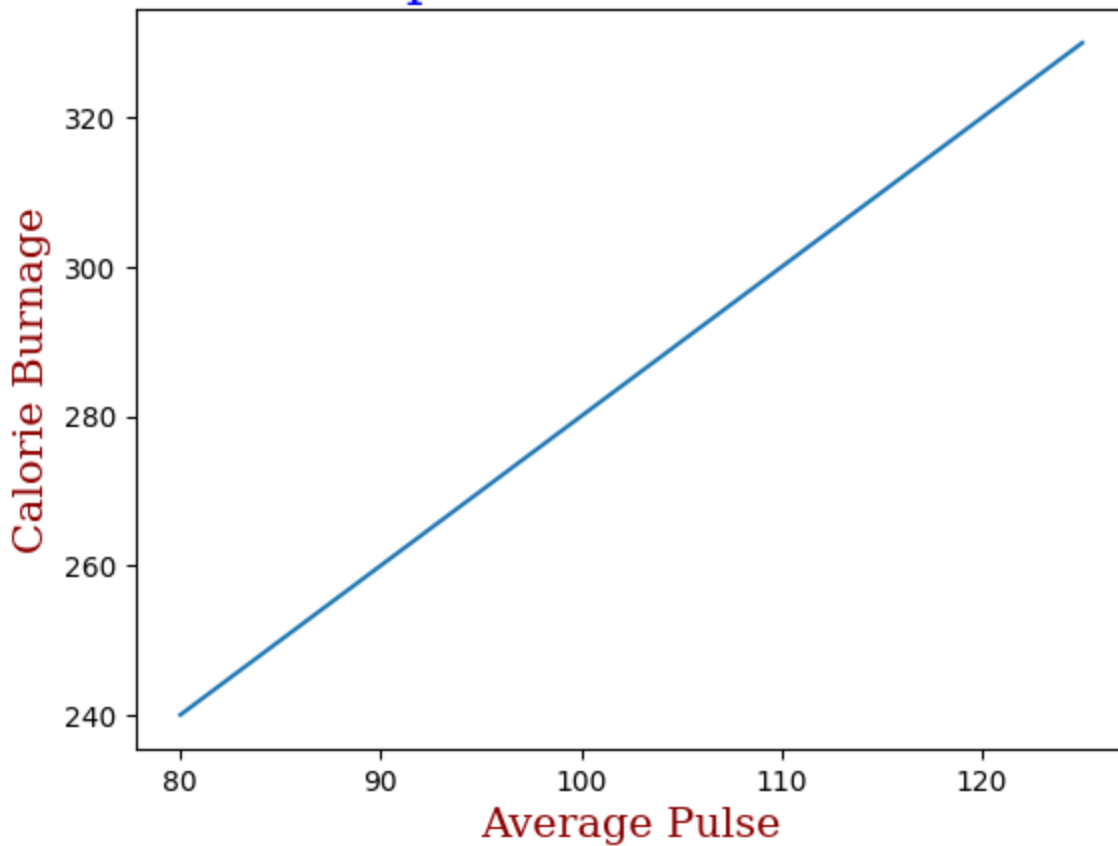
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

font1 = {'family':'serif','color':'blue','size':20}
font2 = {'family':'serif','color':'darkred','size':15}

plt.title("Sports Watch Data", fontdict = font1)
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burnage", fontdict = font2)

plt.plot(x, y)
plt.show()
```

Sports Watch Data



Други видове графики.

В практиката ни се налагат и други начини за визуализиране на информация. Най-популярните класове графики са:

Scatter plot - графика, която визуализира информацията като точки

Bar chart/Bar graph - графика, която представя категоризирани данни с правоъгълни стълбове

Histogram - графика, която представя честотни разпределения

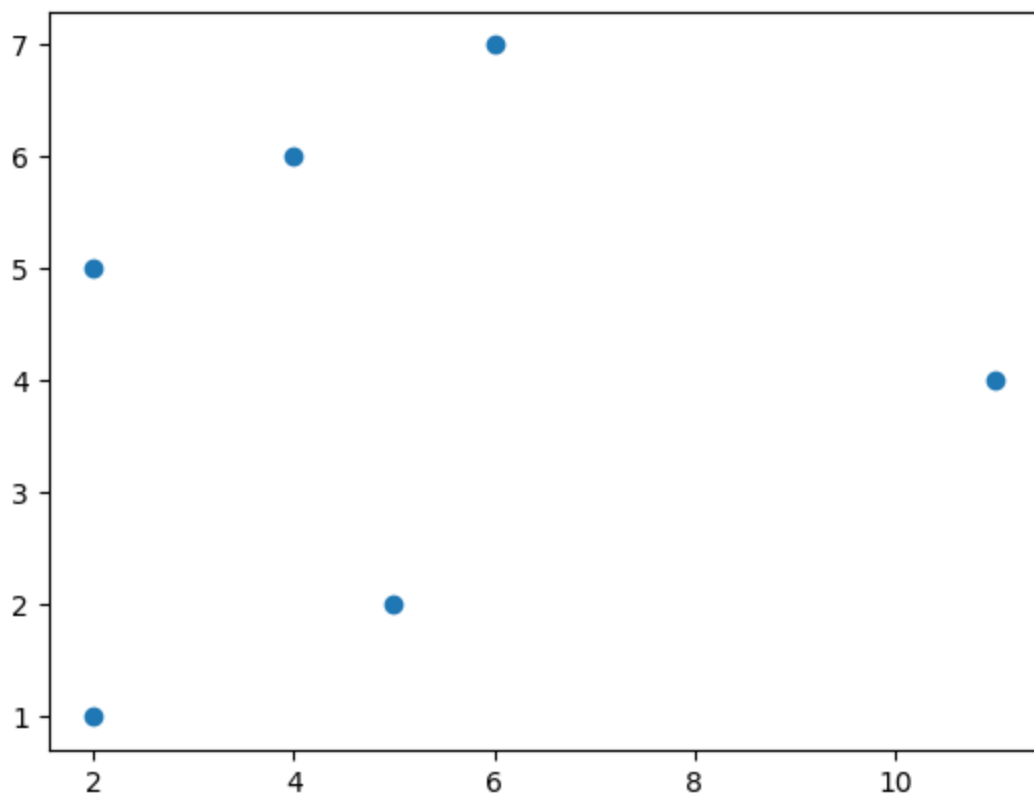
Pie chart - кръгова статистическа графика, която представя числови пропорции

Пример за scatter plot:

```
In [8]: import matplotlib.pyplot as plt
import numpy as np

x_coordinates = np.array((2, 4, 2, 5, 6, 11))
y_coordinates = np.array((1, 6, 5, 2, 7, 4))

plt.scatter(x_coordinates, y_coordinates)
plt.show()
```

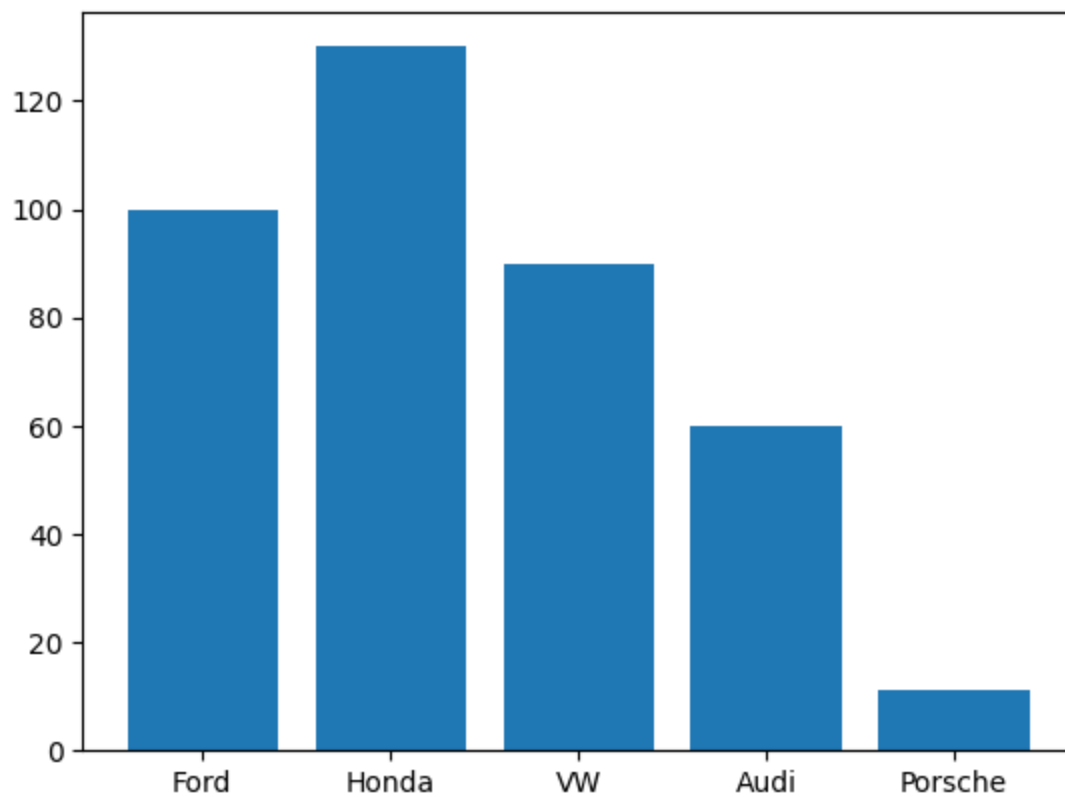


Пример за Bar Chart:

```
In [9]: import matplotlib.pyplot as plt
import numpy as np

x_data = np.array(['Ford', 'Honda', 'VW', 'Audi', 'Porsche'])
y_data = np.array([100, 130, 90, 60, 11])

plt.bar(x_data, y_data)
plt.show()
```

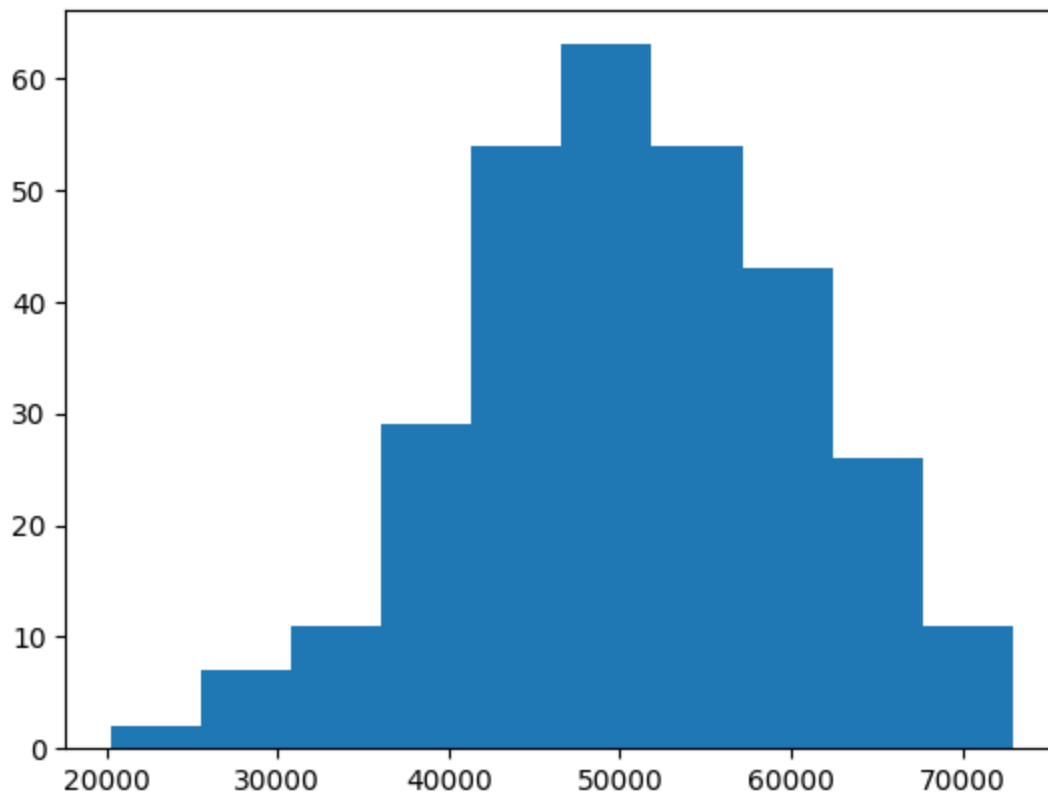


Тъй като хистограмите са честотни графики ще използваме NumPy за да генерираме стойности с нормално разпределение, които ще значат разпределението на годишни заплати на работници в дадена фирма:

```
In [10]: import matplotlib.pyplot as plt
import numpy as np

wages = np.random.normal(50_000, 10_000, 300)

plt.hist(wages)
plt.show()
```

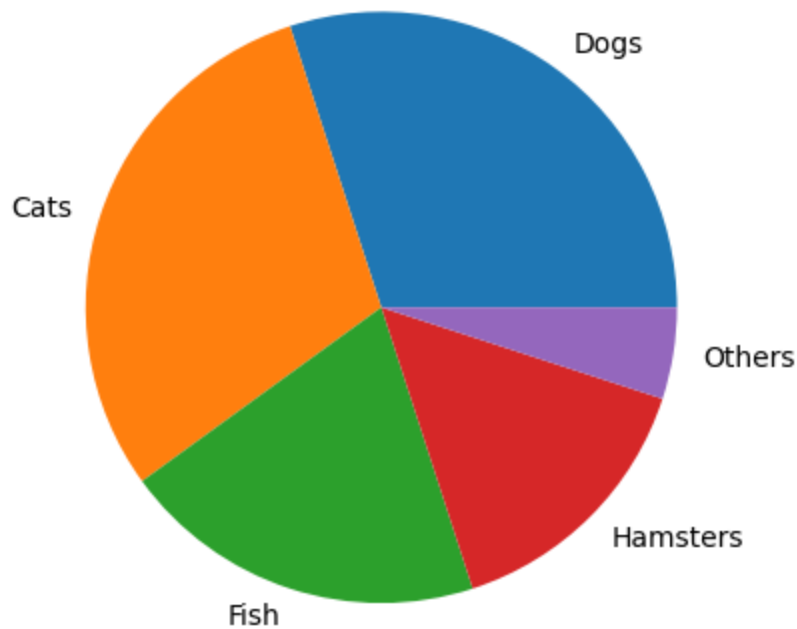


Пример за Pie Chart:

```
In [11]: import matplotlib.pyplot as plt
import numpy as np

pet_data = np.array([30, 30, 20, 15, 5])
pet_data_labels = ['Dogs', 'Cats', 'Fish', 'Hamsters', 'Others']

plt.pie(pet_data, labels=pet_data_labels)
plt.show()
```



Subplots.

В някои случаи ни се налага, за да предаваме контекст на данните си да визуализираме повече от една графика едновременно. Това може да стане със subplot() функцията. Тя приема 3 аргумента, които означават разположението на фигурата.

plt.subplot(1, 3, 1) -> Тази графика има един ред, в който може да имаме 3 графики и тази графика е първата

plt.subplot(1, 3, 3) -> Тази графика има един ред, в който може да имаме 3 графики и тази графика е третата

```
In [12]: import matplotlib.pyplot as plt
import numpy as np

plt.subplot(3, 1, 1)

scatter_x_data = np.array([1, 5, 4, 2])
scatter_y_data = np.array([2, 3, 4, 6])

plt.scatter(scatter_x_data, scatter_y_data)

plt.subplot(3, 1, 2)

bar_x_data = np.array(['Ford', 'Honda', 'VW', 'Audi', 'Porsche'])
bar_y_data = np.array([100, 130, 90, 60, 11])
plt.bar(bar_x_data, bar_y_data)
```

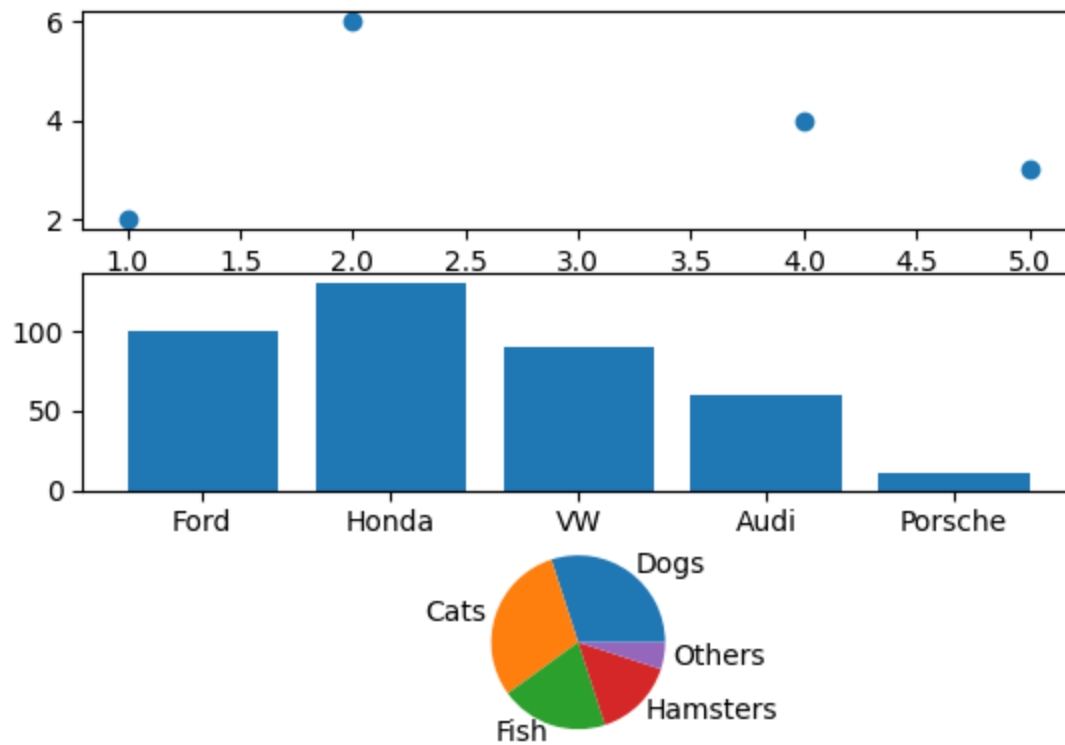
```
plt.subplot(3, 1, 3)
```

```
pie_data = np.array([30, 30, 20, 15, 5])
```

```
pie_data_labels = ['Dogs', 'Cats', 'Fish', 'Hamsters', 'Others']
```

```
plt.pie(pie_data, labels=pie_data_labels)
```

```
plt.show()
```



Самостоятелна задача:

Използвайки dataset-a, който сте си избрали от предния път визуализирайте значима за него информация. Опитайте се да разкажете история с данните, които представяте.